



# Global Snapshots

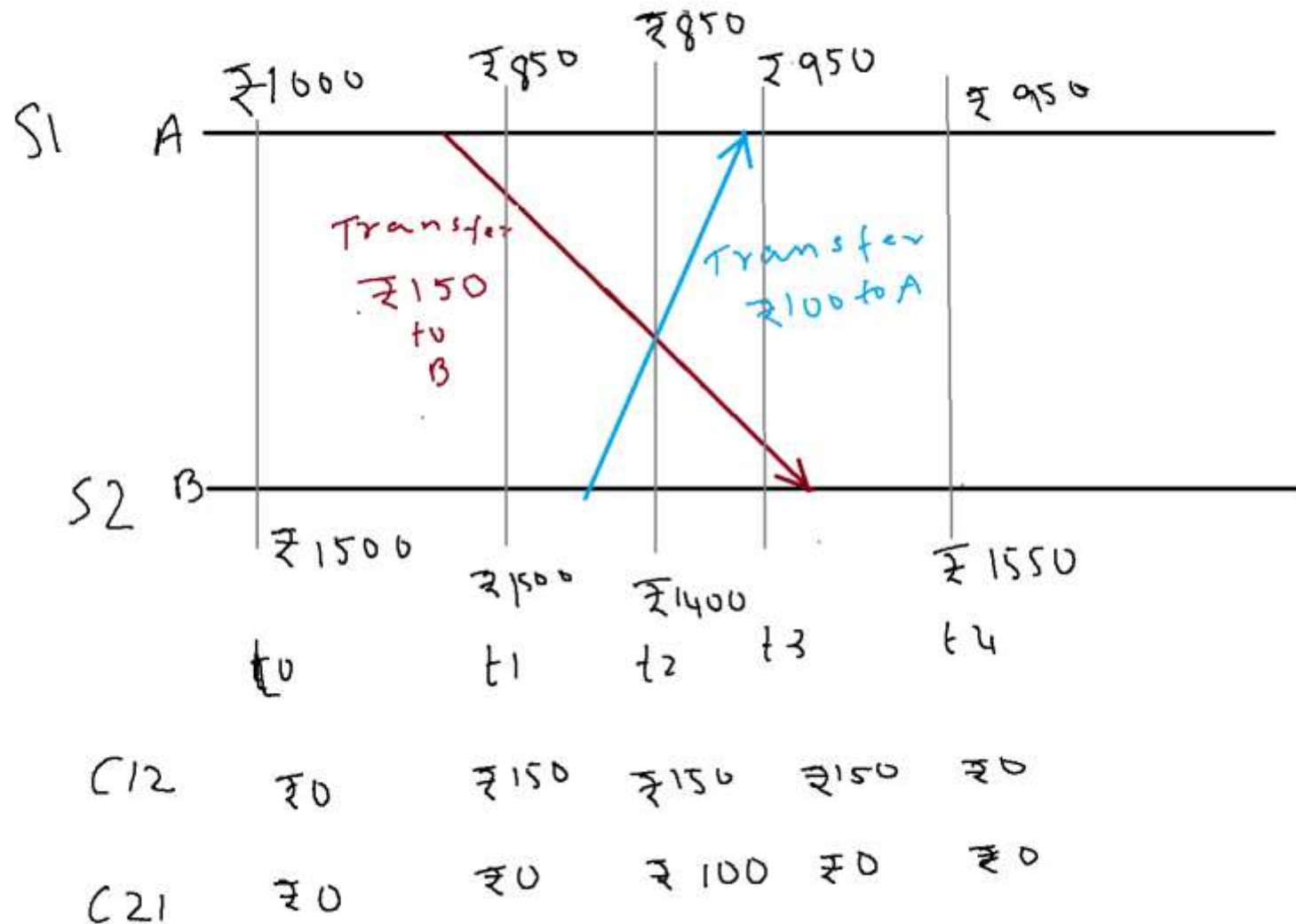
Slides from pallabh Dasgupta

# Global State of a Distributed System

---

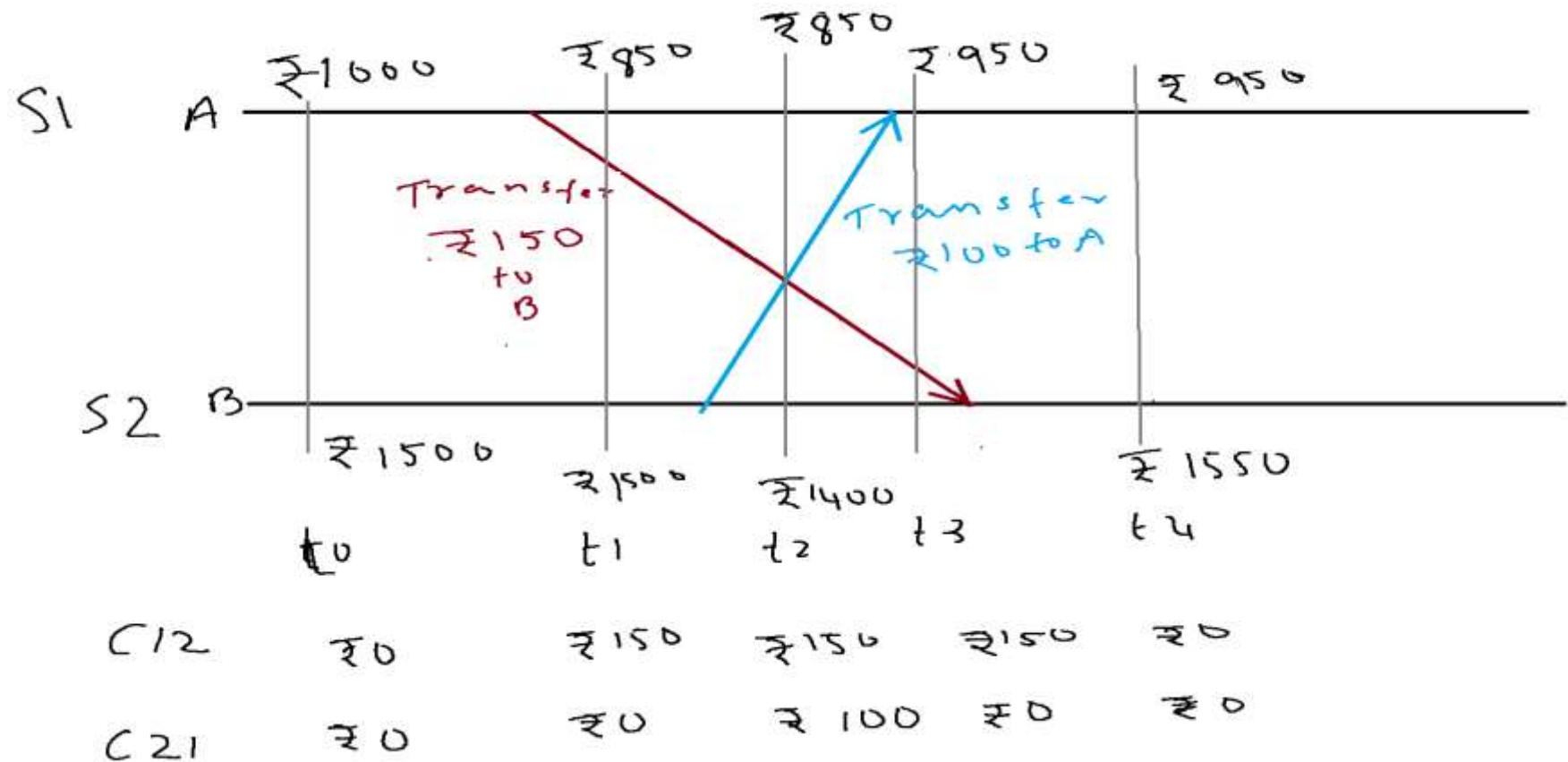
- Cant stop the process and see the global snapshot and neither can we synchronise all the systems and ask them to take a snapshot at a particular instant.
- Not necessary a state the system has visited but is one state that the system could have visited
- Application: Check Pointing Recovery, Deadlock needs understanding current state to measure if a true deadlock.
- Issues: We need to know both node and channel states as some msgs might be in transit
- Assumption: System can not be stopped to take global state. Also no global clock to do the same.

# Global States and Snapshots



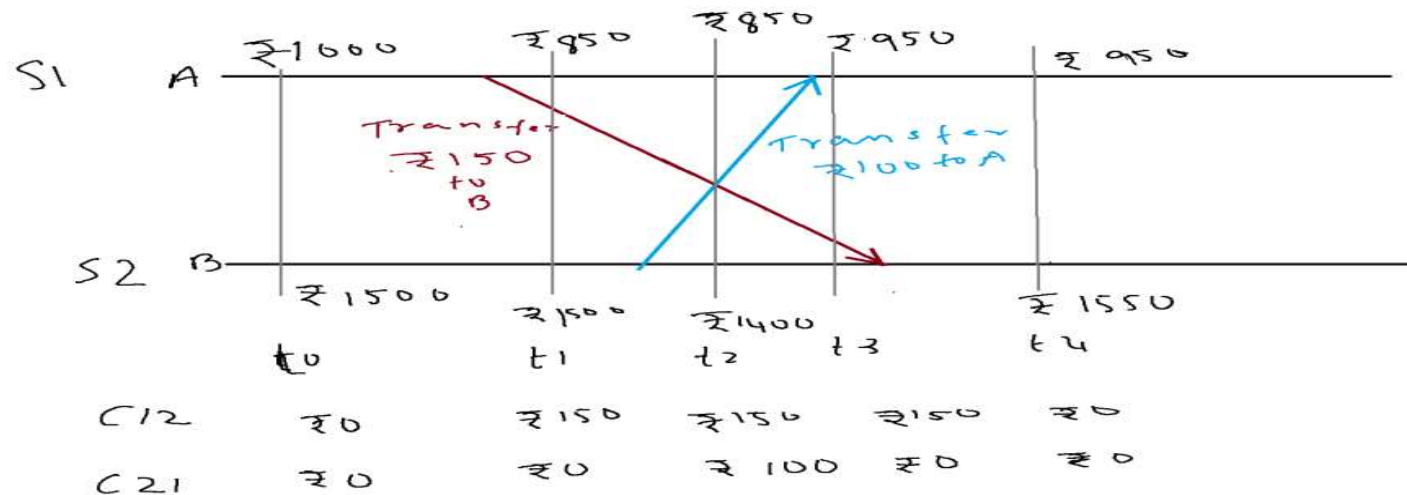
- Consider the distributed execution shown above.

# Global States and Snapshots



- If the state of the account A is recorded at time t0 and the state of account B and that of the channels C12 and C21 are recorded at t2, what is the total money in the system?

# Global States and Snapshots



- Initially  $1000 + 1500 = 2500$
- At check point  $t_0$  for A and  $t_2$  for B we have  
 $1000(A) + 1400(B) + 150(C_{12}) + 100(C_{21}) = 2650$  An extra 150 Rs !!!.
- If this state is used for checkpointing, then there is trouble!

# Global State of a Distributed System

---

- One can think of the collection of local states as the global state.
- Recall that the local state of a process,  $P_i$ , is the contents of the processor registers, stack, local memory, etc.
- The state of a channel is the set of messages in transit in the channel.
- Events lead to changes in the state of local process(es).

# Global States and Snapshots

---

- Recording the global state of a distributed system on-the-fly is an important paradigm.
- The lack of globally shared memory, global clock and unpredictable message delays in a distributed system make this problem non-trivial.
- Building on the definition consistent global states we discuss issues to be addressed to obtain consistent distributed snapshots.
- Then several algorithms to determine on-the-fly such snapshots are presented for several types of networks.

# Global State of a Distributed System

---

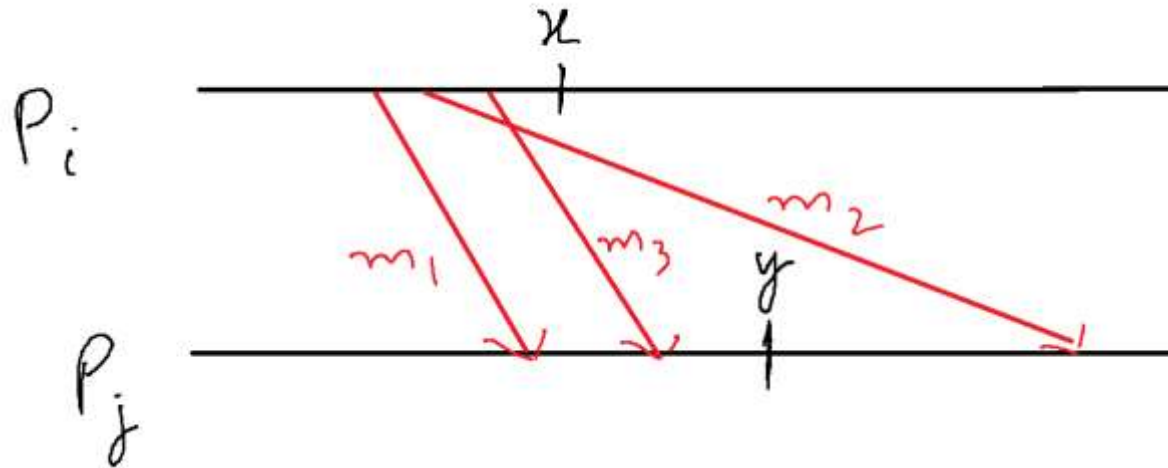
- At any instant the state of a process  $P_i$  denoted by  $LS_i$ , is a result of the sequence of all events executed by  $p_i$  upto that instant. State of a Channel  $C_{ij}$  is denoted as  $SC_{ij}$

The msgs in transit on the channel  $C_{ij}$  are denoted as  
 $SC_{ij} = \{m_{ij} \mid \text{send}(m_{ij}) \in LS_i \text{ and } \text{rec}(m_{ij}) \notin LS_j\}$

- To formalize, let  $LS_i^x$  denote the local state of process  $P_i$  after the occurrence of all events until event  $e_i^x$ 
  - $LS_i^0$  is the initial state of  $P_i$



# Global State of a Distributed System



- We use the above to capture the state of a channel  $C_{ij}$ , denoted  $SC^{x,y}_{i,j}$  as follows.

$$SC^{x,y}_{i,j} = \{m_{ij} \mid \text{send}(m_{ij}) \leq LS^x_i \text{ and } \text{recv}(m_{ij}) \not\leq LS^y_j\}$$

- In words, the state of  $SC^{x,y}_{i,j}$  denotes all messages that have been sent by  $P_i$  up to event  $e^x_i$  and not received by  $P_j$  up to event  $e^y_j$ .

# Conditions for Consistent Global States

---

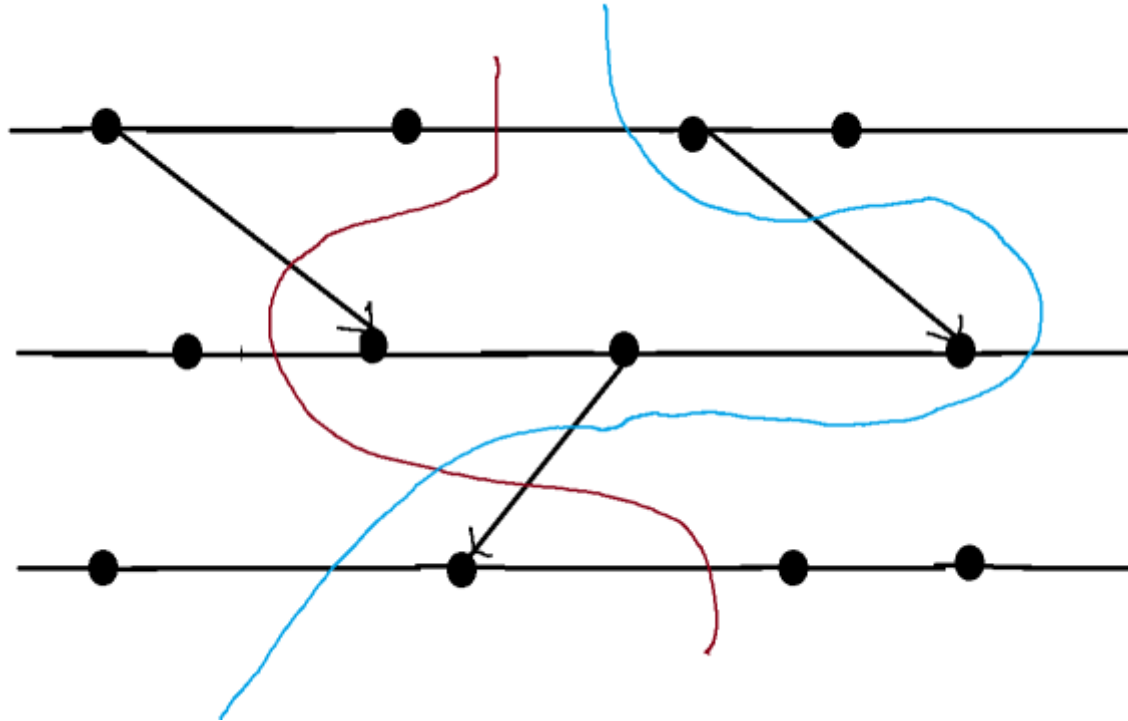
- The global state of a distributed system is a collection of the local states of the processes and the channels.
- Notationally, global state GS is defined as,
  - $GS = \{ \bigcup_i LS_i, \bigcup_{i,j} SC_{ij} \}$
- A global state GS is a **consistent** global state iff it satisfies the following two conditions :  
C1:  $send(m_{ij}) \in LS_i \Rightarrow m_{ij} \in SC_{ij} \oplus rec(m_{ij}) \in LS_j$ . ( $\oplus$  is the Exclusive-OR operator)  
C2:  $send(m_{ij}) \notin LS_i \Rightarrow m_{ij} \notin SC_{ij} \wedge rec(m_{ij}) \notin LS_j$ .

# Conditions for Consistent Global States

---

- A global state GS is a **consistent** global state iff it satisfies the following two conditions :
- C1:  $\text{send}(m_{ij}) \in LS_i \Rightarrow m_{ij} \in SC_{ij} \oplus \text{rec}(m_{ij}) \in LS_j$ . ( $\oplus$  is the Exclusive-OR operator)
- C1 states the law of conservation of messages.
  - Every message that is recorded as sent in the local state of some process is either captured in the state of the channel or is captured in the local state of the receiver.
- C2:  $\text{send}(m_{ij}) \notin LS_i \Rightarrow m_{ij} \notin SC_{ij} \wedge \text{rec}(m_{ij}) \notin LS_j$ .
- C2 states that for every cause there is an effect.
  - If a message is not recorded as sent in the local state of a process  $P_i$ , then the message cannot be included in the state of the channel  $C_{ij}$  or be captured as received by  $P_j$ .

# Interpretation in terms of cuts



- A cut in a space-time diagram is a line joining an arbitrary point on each process line that slices the space-time diagram into a PAST and a FUTURE.
- A consistent global state corresponds to a cut in which every message received in the PAST of the cut was sent in the PAST of that cut.
- Such a cut is known as a **consistent cut**.

# Interpretation in terms of cuts – Example

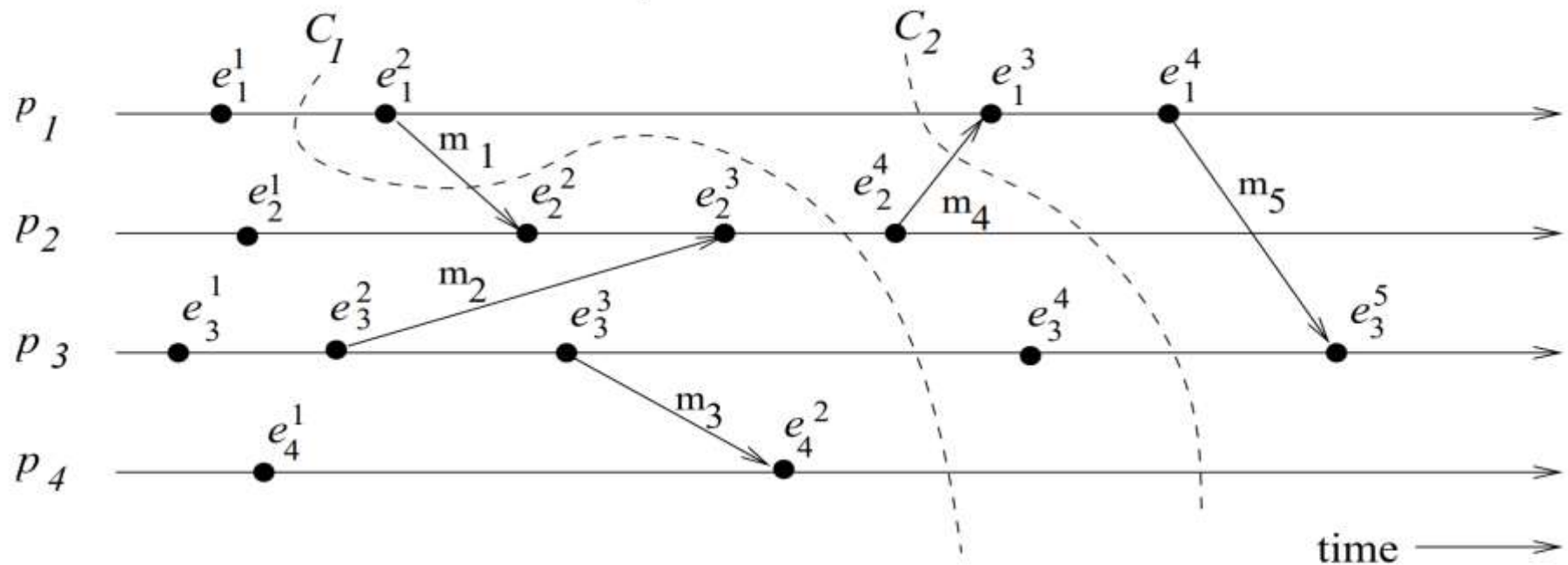
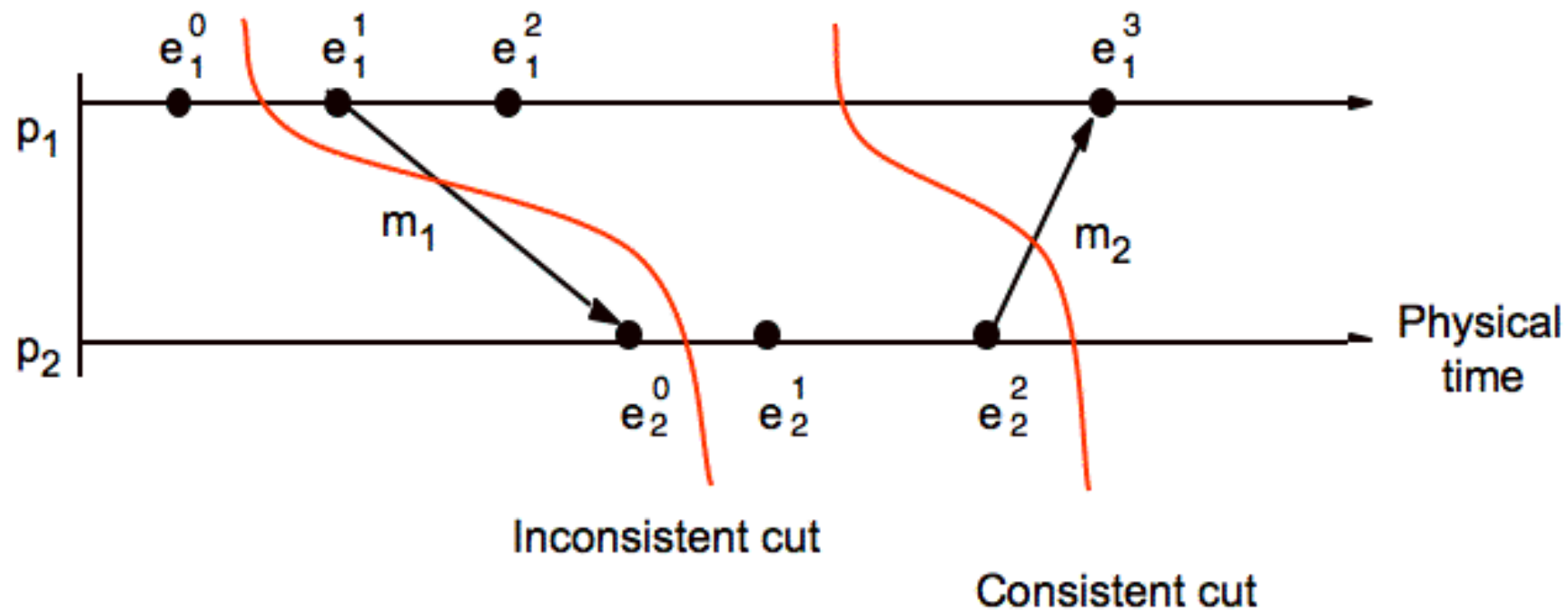


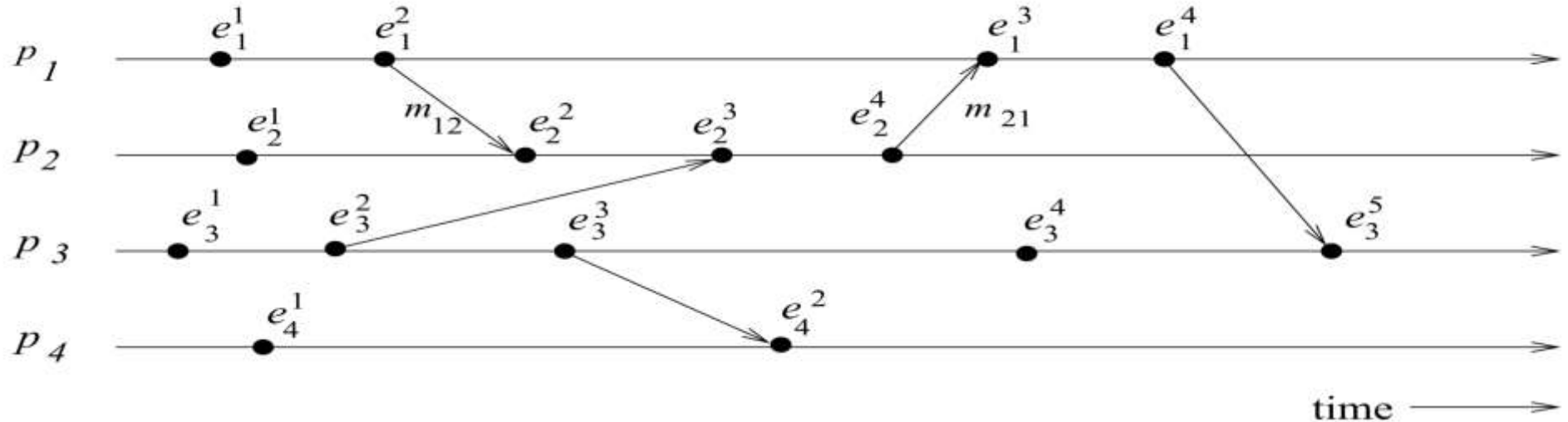
Figure 4.1: An Interpretation in Terms of a Cut.

- Cut  $C_1$  is inconsistent because message  $m_1$  is flowing from the FUTURE to the PAST.
- Cut  $C_2$  is consistent and message  $m_4$  must be captured in the state of channel  $C_{21}$ .

# Consistent and Inconsistent Cut



# Example

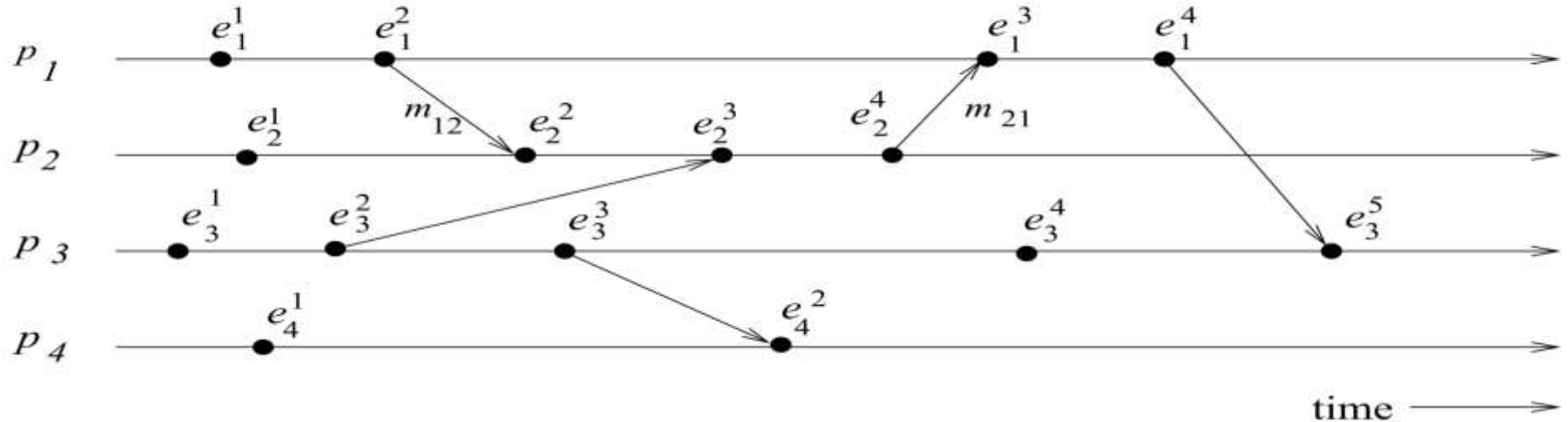


- Which global states are consistent?

- $\{LS_1^1, LS_2^3, LS_3^3, LS_4^2\}$
- $\{LS_1^2, LS_2^4, LS_3^4, LS_4^2\}$
- $\{LS_1^2, LS_2^1, SC_{12}^{22}, LS_3^3, LS_4^2, SC_{32}^{23}\}$

$$SC_{i,j}^{x,y} = \{m_{ij} \mid \text{send}(m_{ij}) \leq LS_i^x \text{ and } \text{recv}(m_{ij}) \not\leq LS_j^y\}$$

# Example



- Which global states are consistent?

1)  $\{LS_1^1, LS_2^3, LS_3^3, LS_4^2\}$  -inconsistent

2)  $\{LS_1^2, LS_2^4, LS_3^4, LS_4^2\}$  - consistent

3)  $\{LS_1^2, LS_2^1, SC_{12}^{22}, LS_3^3, LS_4^2, SC_{32}^{23}\}$  - consistent

$$SC_{i,j}^{x,y} = \{m_{ij} \mid \text{send}(m_{ij}) \leq LS_i^x \text{ and } \text{recv}(m_{ij}) \not\leq LS_j^y\}$$



# Difficulty of Taking a Snapshot

---

- **Issue 1:** How to distinguish between the messages to be recorded in the snapshot from those not to be recorded.
  - Any message that is sent by a process before recording its snapshot, must be recorded in the global snapshot (from C1).
  - Any message that is sent by a process after recording its snapshot, must not be recorded in the global snapshot (from C2).
- **Issue 2:** How to determine the instant when a process takes its snapshot.
  - A process  $p_j$  must record its snapshot before processing a message  $m_{ij}$  that was sent by process  $p_i$  after recording its snapshot.

# A Distributed Program in Execution

---

- Let us now model message delivery.
  - Several ways are possible
    - Non-FIFO : Message can be reordered in transit
    - FIFO : Messages are NOT reordered in transit.
    - Causal : Slightly stronger than FIFO. Messages destined for the same destination are not reordered.
- In symbols,
- for any two messages  $m_{ij}$  and  $m_{kj}$ ,**  
**if  $\text{send}(m_{ij}) \rightarrow \text{send}(m_{kj})$  then  $\text{recv}(m_{ij}) \rightarrow \text{recv}(m_{kj})$ .**
- Causal degenerates to FIFO when sent from same process.  
FIFO  $\subset$  Non-FIFO.



## Global Snapshots for FIFO channels

- Chandy lamports Algorithm

Note:

Each process knows which channel a msg is delivered over.

Consider each process to have an incoming msg channel and outgoing msg channel

# Chandy-Lamport's Algorithm

---

- Uses special marker messages.
- One process acts as initiator, starts the state collection by following the marker sending rule below.
- Marker sending rule for process P:
  - P records its state and
  - For each outgoing channel C from P **on which a marker has not been sent already**, P sends a marker along C before any further message is sent on C

# Chandy Lamport's Algorithm contd..

---

- When Q receives a marker along a channel C:
  - If Q has not recorded its state then Q records its state and also records the state of C as empty; Q then follows the marker sending rule
  - If Q has already recorded its state, it records the state of C as the sequence of messages received along C after Q's state was recorded and before Q received the marker along C

# Chandy Lamport's Algorithm contd..

---

- If Q has already recorded its state, it records the state of C as the sequence of messages received along C after Q's state was recorded and before Q received the marker along C

Say a marker is received by  $P_j$  from say  $P_i$  where  $P_j$  has already recorded its state. Now there can be no messages sent from  $P_i$  to  $P_j$  from when  $P_i$  records the state to when  $P_i$  sends out the markers. Hence this msg received was sent before  $P_i$  marked its state and received after  $P_j$  marked its state. Thus, this msg belongs to the transit messages in Channel  $(P_i, P_j)$

# Notable Points

---

- Markers sent on a channel distinguish messages sent on the channel before the sender recorded its states and the messages sent after the sender recorded its state
- The state collected may not be any state that actually happened in reality, rather a state that “could have” happened

# Process having received a marker

---

Once a process  $P$  receives a marker for the first time he records his state. Lets look at what kinds of msgs he can now receive

- From the initiator – a msg sent before the marker
- From the initiator – the marker msg
- Msg from some process that has not received a marker yet
- Msg from some process that has received its marker and is sending marker to  $P$
- Msg from some process that has received its marker and has already sent a marker to  $P$  and is now sending some other msg



# Notable Points

---

- Markers sent on a channel distinguish messages sent on the channel before the sender recorded its states and the messages sent after the sender recorded its state
- The state collected may not be any state that actually happened in reality, rather a state that “could have” happened
- Will it work if not FIFO channels?
- Message complexity ?

# Notable Points

---

- Markers sent on a channel distinguish messages sent on the channel before the sender recorded its states and the messages sent after the sender recorded its state
- The state collected may not be any state that actually happened in reality, rather a state that “could have” happened
- **Message complexity** ?  $O(|E|)$ , where  $E$  = no. of links

# Chandy-Lamport algorithm – FIFO Channels

---

- This algorithm presented by Mani Chandy and Leslie Lamport works for FIFO channels.
- If channel was not FIFO then the msgs sent after marker would reach before marker and get put into the transit messages list.

# Chandy-Lamport algorithm – FIFO Channels

---

- The Chandy-Lamport algorithm uses a control message, called a **marker** whose role in a FIFO system is to separate messages in the channels.
- After a site has recorded its snapshot, it sends a marker, along all of its outgoing channels before sending out any more messages.
- A marker separates the messages in the channel into those to be included in the snapshot from those not to be recorded in the snapshot.
- A process must record its snapshot no later than when it receives a marker on any of its incoming channels.

# Chandy-Lamport algorithm – FIFO Channels

---

- The algorithm can be initiated by any process by executing the “Marker Sending Rule” by which it records its local state and sends a marker on each outgoing channel.
- A process executes the “Marker Receiving Rule” on receiving a marker.
- If the process has not yet recorded its local state, it records the state of the channel on which the marker is received as empty and executes the “Marker Sending Rule” to record its local state.

# Termination Condition

---

- The algorithm terminates after each process has received a marker on all of its incoming channels.
- All the local snapshots get disseminated to all other processes and all the processes can determine the global state.

# Correctness and Complexity

---

- **Correctness**
- Due to FIFO property of channels, it follows that no message sent after the marker on that channel is recorded in the channel state. Thus, condition C2 is satisfied.
- When a process  $p_j$  receives message  $m_{ij}$  that precedes the marker on channel  $C_{ij}$ , it acts as follows: If process  $p_j$  has not taken its snapshot yet, then it includes  $m_{ij}$  in its recorded snapshot. Otherwise, it records  $m_{ij}$  in the state of the channel  $C_{ij}$ . Thus, condition C1 is satisfied.
- **Complexity**
- The recording part of a single instance of the algorithm requires  $O(e)$  messages.

# Snapshot for non-FIFO Channels

---

- In a non-FIFO system, a marker cannot be used to delineate messages into those to be recorded in the global state from those not to be recorded in the global state.
- In a non-FIFO system, either some degree of inhibition or piggybacking of control information on computation messages to capture out-of-sequence messages.



# The Lai-Yang Algorithm

---

- uses coloring scheme on messages
- Every process is initially white and turns red while taking a snapshot. It then sends a red msg to every other process.
- Every message sent by a white (red) process is colored white (red).
- Thus, a white (red) message is a message that was sent before (after) the sender of that message recorded its local snapshot.
- Every white process takes its snapshot at its convenience, but no later than the instant it receives a red message

# The Lai-Yang Algorithm

---

- Every white process records a history of all white messages sent or received by it along each channel.
- When a process turns red, it sends these histories along with its snapshot to the initiator process that collects the global snapshot.
- The initiator process evaluates  $\text{transit}(LS_i, LS_j)$  to compute the state of a channel  $C_{ij}$ . **HOW ?**

# The Lai-Yang Algorithm

---

- Every white process records a history of all white messages sent or received by it along each channel.
  - When a process turns red, it sends these histories along with its snapshot to the initiator process that collects the global snapshot.
  - The initiator process evaluates  $\text{transit}(LS_i, LS_j)$  to compute the state of a channel  $C_{ij}$  as given below:
  - $SC_{ij}$  = white messages sent by  $p_i$  on  $C_{ij}$  – white messages received by  $p_j$  on  $C_{ij}$
- $$C_{ij} = \{\text{send}(m_{ij}) | \text{send}(m_{ij}) \in LS_i\} - \{\text{rec}(m_{ij}) | \text{rec}(m_{ij}) \in LS_j\}.$$

# The Lai-Yang Algorithm

---

- What is the advantage of Lai-Yangs method over Chandy Laports ?
- Issues ?

- 
- The Lai-Yang Algorithm works in case of non FIFO messages?
  - What about storage?

- 
- The Lai-Yang Algorithm works in case of non FIFO messages?

It does. Channel transmit msgs are white msgs sent before snapshot and have not reached by the time other process has taken its snapshot. They are hence truly in transit and can reach in any order

- Storage
- Storage is heavy as each process has to remember all msgs he has sent received until snapshot

# Snapshots in a Causal Channels

---

- Acharya-Badrinath and Alagar-Venkatesan
- In both these algorithms recording of process state is identical and proceed as follows :
- An initiator process **broadcasts** a token, denoted as token, to every process including itself.
- Let the copy of the token received by process  $p_i$  be denoted  $token_i$ .
- A process  $p_i$  records its local snapshot  $LS_i$  when it receives  $token_i$  and sends the recorded snapshot to the initiator.
- The algorithm terminates when the initiator receives the snapshot recorded by each process.

- 
- How will channel information be captured ?
  - How will global snapshot be captured this way?



# Snapshots in a Causal Channels

---

- The correctness of the approach depends on the following observation.
- For any two processes  $p_i$  and  $p_j$ , the following property is satisfied:
  - $\text{send}(m_{ij}) \notin LS_i \Rightarrow \text{rec}(m_{ij}) \notin LS_j$ .  
This is because a send after process  $i$  received the token can reach process  $j$  only after process  $j$  received the token and hence would have marked its state (by causal property as the instant of token broadcast was before the send msg to process  $j$ ).
- Channel state recording is done differently in different algorithms. Check Acharya-Badrinath and Alagar-Venkatesan for examples. Their task is to identify msgs sent before the snapshot  $P_i$  and received after the snapshot of  $P_j$  as transit for  $C_{ij}$

# Acharya – Badrinath Algorithm

---

When we assume causal ordering then will there be transit msgs ?

# Acharya – Badrinath Algorithm

---

How can I measure the transit messages?

Hint: Each process will keep track of number of msgs sent/received by it to every other process.

# Acharya – Badrinath Algorithm

---

- Each process  $p_i$  maintains arrays  $SENT_i[1, \dots, N]$  and  $RECD_i[1, \dots, N]$ .
- $SENT_i[j]$  is the number of messages sent by process  $p_i$  to process  $p_j$ .
- $RECD_i[j]$  is the number of messages received by process  $p_i$  from process  $p_j$ .
- Channel states are recorded as follows:
- When a process  $p_i$  records its local snapshot  $LS_i$  on the receipt of token, it includes arrays  $RECD_i$  and  $SENT_i$  in its local state before sending the snapshot to the initiator.

# Acharya – Badrinath Algorithm

---

- When the algorithm terminates, the **initiator** determines the state of channels as follows:
- The state of each channel from the initiator to each process is empty.
- The state of channel from process  $p_i$  to process  $p_j$  is the set of messages whose sequence numbers are given by  $\{RECD_j[i] + 1, \dots, SENT_i[j]\}$ .

That is, sent 10 msgs but received 5 msgs so 6,7,8,9,10 are not received. Due to causal order they will appear at receiving end in that order itself.

## Complexity:

- This algorithm requires  $2n$  messages